Draw a simple trace: which values of row and col are visited first, and how many cells are printed?

What did we discover?

CODE:

[0,0] → [0,1] → [0,2]

[1,0] → [1,1] → [1,2]

[2,0] → [2,1] → [2,2]

There are nine cell visits. The outer loop moves down, the inner loop moves across.

IMPROVE: Explain one correction to your own reasoning.

DO NOW — RECALL

Noughts & Crosses · Part 2 — Discover iteration

Name: _____ Class: _____ Date: _____

1. In a zero-indexed C# array, which element does array[2] access?

2. Name the three parts of a C# for loop header used to visit every array element.

3. In a C# 2D array organised as rows and columns, what does GetLength(0) return?

4. If a board has 3 rows and 3 columns, how many values does it contain?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

The item at index 2 — the third item.

2. Model answer

A counter, condition and update.

3. Model answer

The number of rows.

4. Model answer

9.

By the end, you can…

• spot repeated row-printing instructions and replace them with a loop;

• explain why a loop inside another loop is useful for a 2D board;

• create a reusable DrawBoard() method using GetLength() .

Vocabulary — know these terms

Recall the term — 2D array

Recall the term — row

Recall the term — column

Recall the term — index

Recall the term — dimension

Recall the term — nested loop

Recall the term — GetLength()

Recall the term — traverse

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

What did we discover?

The code prints three values separated by pipes on every row. Only the row index changes (0 → 1 → 2).

IMPROVE: Explain one correction to your own reasoning.

Guided 2 — original CHECK

Write or predict

Complete the counter condition to visit rows 0, 1 and 2 in sequence.

CODE:

for (int row = 0; row < ?; row++)

What did we discover?

CODE:

for (int row = 0; row < board.GetLength(0); row++)

Using the row count is better than assuming every board has three rows.

IMPROVE: Explain one correction to your own reasoning.

Guided 3 — original CHECK

Write or predict

For one row, write a loop that visits every column. Which dimension returns the number of columns?

CODE:

for (int col = 0; col < ?; col++)

What did we discover?

CODE:

for (int col = 0; col < board.GetLength(1); col++)

The inner loop visits the columns across a single row.

IMPROVE: Explain one correction to your own reasoning.

Guided 4 — original CHECK

Write or predict

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

board.GetLength(1) returns the number of columns in the board.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

NEW LEARNING

Read, annotate and explain

Yesterday: three similar instructions

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

The problem: repeated code is harder to maintain. Today we will discover why iteration makes the same board easier to draw.

Notice the repeated statements

Look at the three explicit row-printing instructions. The operation is the same; only the row index changes: 0, 1, 2 .

Replace the rows with one loop

The first loop repeats the same row-print statement. row changes automatically; each repetition chooses a different row.

Spot the second repetition

Inside each row, we still refer to columns 0, 1 and 2. A second loop can move across the columns of one row.

Put one loop inside the other

Outer loop = down the rows. Inner loop = across the columns. This is called nested iteration .

Make it reusable

GetLength(0) gives the row count; GetLength(1) gives the column count. Place the loops in DrawBoard() , then call it whenever the display needs refreshing.

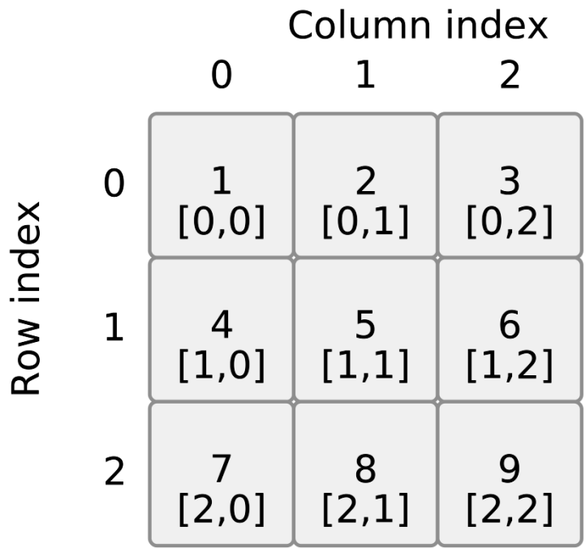
Keep the board array as a class-level field so DrawBoard() can access it. Lesson 015 uses the same board for moves.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Yesterday: three similar instructions



Numbered Noughts and Crosses board with zero-based row and column indexes.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Replace repeated columns
Write a for loop to display one row, visiting columns using col . Print separators only between squares.

Question 4

Build DrawBoard()
Combine your two loops to visit all nine squares and put them inside DrawBoard() . Keep board accessible from the method and call it in Main() .

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Start with what worked
Use your three row-print statements from Lesson 014. Run them unchanged.

Question 2

Replace repeated rows
Write one for loop that prints all three rows. Within the loop you may still access columns 0, 1 and 2 explicitly.

I DO — WORKED EXAMPLE

Follow the worked method

From three statements to one loop, then two

Start with the old solution

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

It works. But it repeats the same pattern three times.

Replace the three row statements

```
for (int row = 0; row < board.GetLength(0); row++)
{
    Console.WriteLine(board[row,0] + " | " +
                      board[row,1] + " | " +
                      board[row,2]);
}
```

Keep the row output the same but allow row to change inside the loop.

Replace the repeated columns

```
for (int col = 0; col < board.GetLength(1); col++)
{
    Console.Write(board[0,col]);
    if (col < board.GetLength(1)-1)
        Console.Write(" | ");
}
Console.WriteLine();
```

Now each column is selected by col . After the third column, finish the row.

Combine both loops

```
for (int row = 0; row < board.GetLength(0); row++)
{
    for (int col = 0; col < board.GetLength(1); col++)
    {
        Console.Write(board[row,col]);
        if (col < board.GetLength(1)-1)
            Console.Write(" | ");
    }
    Console.WriteLine();
}
```

Predict how many times Console.Write(board[row,col]) executes: 9 .

Refine into DrawBoard()

```
public static void DrawBoard()
{
    for (int row = 0; row < board.GetLength(0); row++)
    {
        for (int col = 0; col < board.GetLength(1); col++)
        {
            Console.Write(board[row,col]);
            if (col < board.GetLength(1)-1)
                Console.Write(" | ");
        }
        Console.WriteLine();
    }
}
```

This method assumes board is stored at class level. Call DrawBoard() from Main() . You can reuse it after every move in Lesson 015.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Identify the pattern

The original problem stays visible as we improve the code.

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

Method / working:

2. Introduce a row loop

The original problem stays visible as we improve the code.

```
for (int row = 0; row < ?; row++)
```

Method / working:

3. Notice the columns

The original problem stays visible as we improve the code.

```
for (int col = 0; col < ?; col++)
```

Method / working:

4. Combine and explain

The original problem stays visible as we improve the code.

Method / working:

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: Identify the pattern The original problem stays visible as we improve the code. Write or predict Study the original three row-print statements. Which part remains identical and which…

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 2: Introduce a row loop The original problem stays visible as we improve the code.

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 3: Notice the columns The original problem stays visible as we improve the code.

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

Guided 4: Combine and explain The original problem stays visible as we improve the code.

DISCUSS — Compare approaches, walk through one test input, and explain how you would detect or correct a mistake.

IMPROVE — Write one precise correction or stronger explanation in your own answer.